CSL7640: Natural Language Understanding - Assignment 2

Student: `Shashank Parchure`
Roll No: B23CM1059

GitHub Repository Link for Assignment 2

1 Problem 1: Learning word embeddings from IIT Jodhpur data

1.1 Objective

Train Word2Vec models (CBOW and Skip-gram with Negative Sampling) on an IIT Jodhpur corpus and analyze the semantic structure captured by learned embeddings using similarity queries, analogy tests, and 2D visualization.

1.2 Corpus Curation and Preprocessing

Sources used (at least 3):

- IIT Jodhpur official website pages (departments, programs, research, announcements)
- Academic regulation documents (mandatory)
- Additional institute documents such as course/syllabus pages and circulars/newsletters

Preprocessing steps:

1. Collect raw text from the above sources and keep only English content.
2. Remove boilerplate/non-content text (headers, footers, menus, repeated navigation blocks).
3. Normalize whitespace and formatting artifacts (extra line breaks, stray symbols).
4. Lowercase all text.
5. Remove excessive punctuation/non-textual artifacts; keep clean word tokens.
6. Tokenize into words and store the cleaned corpus as `iitj_corpus_clean.txt`.
7. During training, apply a minimum frequency filter (default used: `min_freq=3`) to drop very rare words.

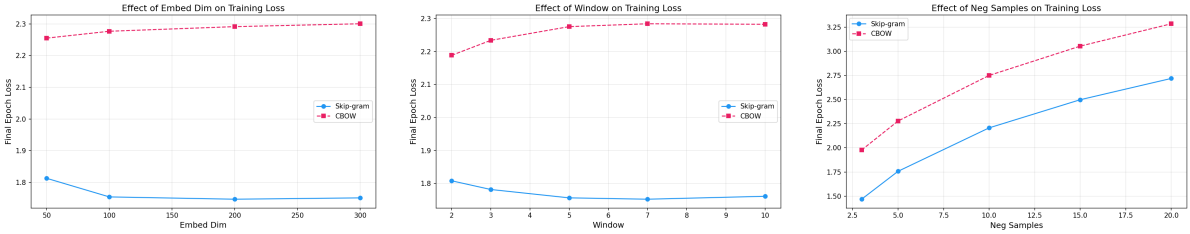
Dataset statistics (final cleaned corpus):

- Total Documents: 38
- Total Tokens: 179990
- Vocabulary Size: 17047

1.3 Model Training (From Scratch)

Two models were implemented using NumPy:

- **Skip-gram with Negative Sampling (SGNS)**
- **CBOW with Negative Sampling**



(a) Embedding dim sweep (b) Window sweep (c) Negative samples sweep

Figure 1: Training-loss trends for CBOW vs. Skip-gram under hyperparameter sweeps.

1.3.1 Negative Sampling Objective

Let $\mathbf{v}_c$ be the input (center/context) embedding and $\mathbf{u}_w$ be the output embedding of word w.

Skip-gram (center → context):

$$J = -\log \sigma(\mathbf{u}_o^\top \mathbf{v}_c) - \sum_{k=1}^{K} \log \sigma(-\mathbf{u}_{n_k}^\top \mathbf{v}_c)$$

CBOW (context avg → target):

$$\hat{\mathbf{v}} = \frac{1}{|C|} \sum_{j \in C} \mathbf{v}_j, \quad J = -\log \sigma(\mathbf{u}_o^\top \hat{\mathbf{v}}) - \sum_{k=1}^{K} \log \sigma(-\mathbf{u}_{n_k}^\top \hat{\mathbf{v}})$$

Negative words were sampled from:

$$P(w) \propto U(w)^{3/4}$$

where $U(w)$ is the unigram frequency distribution. Optimization used SGD with a linearly decayed learning rate.

1.3.2 Hyperparameter Sweeps

We trained both models while varying one hyperparameter at a time (others fixed):

- Embedding dimension: $\{50, 100, 200, 300\}$
- Context window size: $\{2, 3, 5, 7, 10\}$
- Negative samples: $\{3, 5, 10, 15, 20\}$

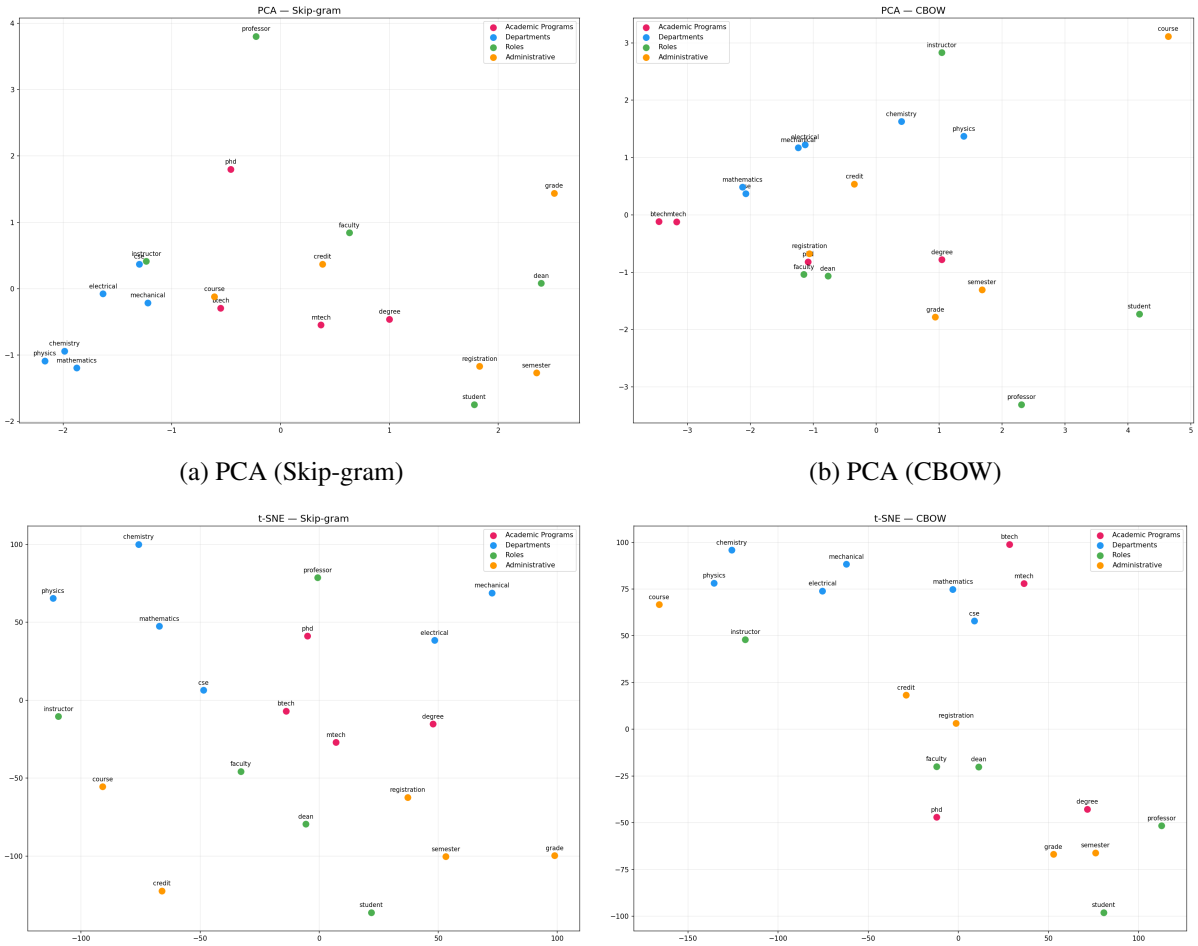
1.4 Semantic Analysis

Cosine similarity was used to find nearest neighbors:

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x}^\top \mathbf{y}}{\|\mathbf{x}\| \, \|\mathbf{y}\|}.$$

1.4.1 Nearest Neighbors

Nearest neighbors were reported for: `research`, `student`, `phd`, `exam`, `course`. (Outputs are generated by `semantic_analysis.py`; results vary slightly across hyperparameters and model type.)



(a) PCA (Skip-gram)

(b) PCA (CBOW)

(c) t-SNE (Skip-gram)

(d) t-SNE (CBOW)

Figure 2: 2D projections of selected words. t-SNE typically shows more visually separated clusters; PCA preserves global linear structure.

1.4.2 Analogy Experiments

Analogies were computed using the standard vector arithmetic:

$$\mathbf{v} = \mathbf{e}(b) - \mathbf{e}(a) + \mathbf{e}(c),$$

and the nearest words to $\mathbf{v}$ were returned (excluding a, b, c).

Analogy prompts used:

- `Undergraduate : BTech :: Postgraduate : ?`
- `dean : office :: hod : ?`

Observation: Skip-gram typically provides sharper semantic neighborhoods for less frequent academic terms, while CBOW tends to produce smoother embeddings for very frequent words. Meaningfulness of analogies depends strongly on corpus coverage of the involved terms.

1.5 2D Visualization (PCA and t-SNE)

To interpret embedding structure, selected domain words were projected to 2D using PCA and t-SNE (sklearn). Clusters were visualized for groups such as academic programs, departments, roles, and administrative terms.

2 Problem 2: Character-level name generation using RNN variants

2.1 Objective

The goal is to model Indian given names at the character level and generate new names. Three sequence models were implemented and compared: (i) Vanilla RNN, (ii) Bidirectional LSTM (BLSTM), and (iii) RNN with a basic attention mechanism. The dataset `TrainingNames.txt` contains 1000 lowercase names, one per line. Names are modeled as sequences with special tokens `<s>` (start), `</s>` (end), and `<pad>` (padding for batching).

2.2 Task 1: Model Implementation

2.2.1 Vanilla RNN Language Model

Architecture: Characters are embedded into vectors and fed into a single-layer recurrent cell:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{ih}\mathbf{e}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1})$$

A linear projection maps $\mathbf{h}_t$ to logits over the character vocabulary.

Training objective: next-character prediction with cross-entropy loss, ignoring padded positions.

Hyperparameters used (default run): embedding dim $= 64$, hidden size $= 128$, batch size $= 64$, epochs $= 30$, optimizer AdamW with learning rate 3×10^{-3}, gradient clipping $= 1.0$.

Trainable parameters: reported directly by the training script.

2.2.2 Bidirectional LSTM (BLSTM)

Architecture: A custom LSTM cell is implemented and run in both forward and backward directions. For each time step, forward and backward hidden states are concatenated and projected to vocabulary logits:

$$\mathbf{h}_t = [\mathbf{h}_t^{(f)}; \mathbf{h}_t^{(b)}], \quad \mathbf{o}_t = \mathbf{W}\mathbf{h}_t + \mathbf{b}$$

Training objective: next-character prediction with cross-entropy loss (padding ignored). Since the model is bidirectional, it can leverage both left and right context during training.

Generation detail: during sampling, the BLSTM is used by appending a fixed `future_window` of padding tokens after the current prefix; the next character distribution is taken from the last prefix position. A minimum length constraint was used during decoding to avoid immediate termination.

Hyperparameters used (default run): embedding dim $= 64$, hidden size $= 128$ per direction, batch size $= 64$, epochs $= 30$, AdamW with learning rate 3×10^{-3}, `future_window`$= 16$.

Trainable parameters: reported directly by the training script.

2.2.3 RNN with Basic Attention

Architecture: A vanilla RNN produces hidden states $\mathbf{h}_1, \ldots, \mathbf{h}_t$. At each time step, a dot-product attention is computed over all previous hidden states (including current), producing a context vector:

$$\alpha_{t,i} = \mathrm{softmax}\left(\frac{\mathbf{q}_t^\top \mathbf{k}_i}{\sqrt{d}}\right), \quad \mathbf{c}_t = \sum_{i=1}^{t} \alpha_{t,i}\mathbf{v}_i$$

The hidden state and context are combined and projected to character logits.

Training objective: next-character prediction with cross-entropy loss, ignoring padded positions.

Hyperparameters used (default run): embedding dim $= 64$, hidden size $= 128$, batch size $= 64$, epochs $= 30$, AdamW with learning rate 3×10^{-3}.

Trainable parameters: reported directly by the training script.

Model	Generated (N)	Novelty (%)	Diversity
Vanilla RNN	100	84.00	1.000
BLSTM	100	99.00	0.560
RNN + Attention	100	69.00	1.000

Table 1: Novelty and diversity on generated names (100 samples per model).

2.3 Task 2: Quantitative Evaluation

For each model, 100 names were generated and evaluated using:

Novelty Rate: percentage of generated names not present in the training set.

$$\text{Novelty} = \frac{\#\{g \notin \mathcal{T}\}}{\#\{g\}} \times 100$$

Diversity: unique generated names divided by total generated names.

$$\text{Diversity} = \frac{\#\text{unique}(g)}{\#\{g\}}$$

2.3.1 Results

Interpretation: The BLSTM attains very high novelty, but its diversity is lower, indicating repeated patterns in the generated outputs. The vanilla RNN and attention model both achieve perfect diversity on this sample (all 100 generated names are unique), with the vanilla RNN achieving higher novelty than the attention model.

2.4 Qualitative Analysis

In addition to novelty and diversity, the readability and "name-likeness" of generated samples were inspected using the provided output files.

Vanilla RNN: The model produces names with reasonable length and common phonetic patterns. Several samples look close to plausible names while still being novel, for example `kavika`, `adwitesh`, `aadhava`, and `subindiri`. Occasional odd spellings appear (e.g., unusual consonant clusters), but overall the outputs remain fairly consistent with the training distribution.

RNN + Attention: The attention model often generates fluent names and tends to reuse recognizable fragments from training data (e.g., `gangadhar`, `rudranil`, `tamizhan`, `nishikant`). This aligns with its lower novelty compared to the vanilla RNN. Despite that, the samples remain varied and generally well-formed.

BLSTM: The BLSTM outputs are predominantly very short (often 2–3 characters) with frequent consonant pairs (e.g., `lo`, `ch`, `sh`, `kh`, `zz`). Even though the novelty score is high, many generated strings are not name-like. This highlights a practical limitation of using bidirectional sequence models for left-to-right generation: the model is trained to exploit both past and future context, while decoding must operate without true future characters.

2.5 Summary

Across the three variants, the vanilla RNN provides the best balance between novelty and qualitative name-likeness on this dataset. The attention model generates fluent outputs but exhibits lower novelty due to partial reuse of training-set substrings. The BLSTM achieves very high novelty in terms of string matching, but the generated outputs are often short and repetitive, which reduces perceived quality despite the strong novelty metric.